

Análisis

Ejercicio 3

Requisitos funcionales

1. Registrar una cantidad dinámica de órdenes de servicio sin establecer un límite fijo.
2. Almacenar las órdenes utilizando una colección declarada como List< y creada mediante ArrayList.
3. Validar al registrar: número de orden no repetido, propietario no vacío, placa no vacía, descripción no vacía y costo estimado mayor que 0.
4. Consultar todas las órdenes registradas actualmente.
5. Buscar una orden por su número de orden y mostrar toda su información.
6. Informar cuando una orden buscada no exista sin finalizar el programa.
7. Modificar la descripción del servicio y el costo estimado de una orden existente.
8. Cancelar una orden eliminándola de la colección dinámica.
9. Consultar todas las órdenes asociadas a una placa específica.
10. Calcular el valor total de las órdenes activas.
11. Calcular el costo promedio de las órdenes activas.
12. Determinar la orden con el costo estimado más alto.
13. Indicar la cantidad actual de órdenes registradas.
14. Manejar entradas incorrectas mediante try-catch, especialmente datos numéricos inválidos.
15. Manejar mediante try-catch al menos una búsqueda, modificación o cancelación de una orden inexistente.
16. Utilizar finally para ejecutar una acción tanto si ocurre una excepción como si no.
17. Después de manejar una excepción, regresar al menú principal y continuar la ejecución.
18. Mostrar un menú con: registrar, consultar, buscar, modificar, cancelar, consultar por placa, reporte de costos, mayor costo, cantidad y salir.

Clases identificadas y propósito

Clase	Propósito
OrdenServicio	Representa una orden individual y guarda los datos propios del vehículo, propietario, servicio y costo.
GestorOrdenes	Administra la colección dinámica de órdenes y concentra las operaciones de agregar, buscar, modificar, eliminar y calcular reportes.
Vista	Se encarga de la interacción por consola: mostrar menú, mensajes, órdenes y reportes, además de solicitar datos al usuario.
Controlador	Coordina las opciones del menú, comunica la Vista con el GestorOrdenes y aplica try-catch/finally en las operaciones que pueden fallar.
Main	Driver program. Crea las instancias principales y llama al controlador para iniciar el sistema.

Atributos

Tipo de dato	Variable	Visibilidad	Propósito	Clase
int	numeroOrden	private	Identificador único de la orden.	OrdenServicio
String	nombrePropietario	private	Guarda el nombre del propietario.	OrdenServicio
String	placaVehiculo	private	Guarda la placa del vehículo.	OrdenServicio
String	descripcionServicio	private	Describe el mantenimiento o reparación solicitada.	OrdenServicio
double	costoEstimado	private	Guarda el costo estimado; debe ser mayor que 0.	OrdenServicio
List<OrdenServicio>	ordenes	private	Colección dinámica que almacena las órdenes activas.	GestorOrdenes
Scanner	sc	private	Permite leer las entradas del usuario desde consola.	Vista
GestorOrdenes	gestor	private	Referencia al modelo que administra las órdenes.	Controlador
Vista	vista	private	Referencia a la vista usada para entrada y salida.	Controlador

4. Métodos

Método	Retorno	Visibilidad	Parámetros	Propósito	Clase
OrdenServicio()	—	public	int numeroOrden, String nombrePropietario, String placaVehiculo, String descripcionServicio, double costoEstimado	Inicializa una orden con todos sus datos.	OrdenServicio
getNumeroOrden()	int	public	Ninguno	Devuelve el número de orden.	OrdenServicio
getNombrePropietario()	String	public	Ninguno	Devuelve el nombre del propietario.	OrdenServicio
getPlacaVehiculo()	String	public	Ninguno	Devuelve la placa del vehículo.	OrdenServicio
getDescripcionServicio()	String	public	Ninguno	Devuelve la descripción del servicio.	OrdenServicio
getCostoEstimado()	double	public	Ninguno	Devuelve el costo estimado.	OrdenServicio
setDescripcionServicio()	void	public	String descripcionServicio	Actualiza la descripción; rechaza texto vacío.	OrdenServicio

setCostoEstimado()	void	public	double costoEstimado	Actualiza el costo; rechaza valores <= 0.	OrdenServicio
GestorOrdenes()	—	public	Ninguno	Inicializa ordenes = new ArrayList<>();	GestorOrdenes
agregarOrden()	void	public	OrdenServicio orden	Agrega una orden si su número no está repetido.	GestorOrdenes
obtenerOrdenes()	List<OrdenServicio>	public	Ninguno	Devuelve las órdenes activas para consultarlas.	GestorOrdenes
buscarOrden()	OrdenServicio	public	int numeroOrden	Busca por número; si no existe lanza NoSuchElementException.	GestorOrdenes
modificarOrden()	void	public	int numeroOrden, String descripcion, double costo	Busca y actualiza descripción y costo.	GestorOrdenes
cancelarOrden()	void	public	int numeroOrden	Busca y elimina la orden de la colección.	GestorOrdenes
buscarPorPlaca()	List<OrdenServicio>	public	String placa	Devuelve todas las órdenes cuya placa coincide.	GestorOrdenes
calcularTotal()	double	public	Ninguno	Suma los costos de todas las órdenes activas.	GestorOrdenes
calcularPromedio()	double	public	Ninguno	Calcula total / cantidad; devuelve 0 si no hay órdenes.	GestorOrdenes
obtenerOrdenMayorCosto()	OrdenServicio	public	Ninguno	Recorre la lista y devuelve la orden con mayor costo; si está vacía lanza NoSuchElementException.	GestorOrdenes
cantidadOrdenes()	int	public	Ninguno	Devuelve ordenes.size().	GestorOrdenes
Vista()	—	public	Ninguno	Inicializa el Scanner.	Vista
mostrarMenu()	void	public	Ninguno	Muestra las 10 opciones del sistema.	Vista
leerEntero()	int	public	String mensaje	Solicita y convierte un entero; una entrada inválida puede producir NumberFormatException.	Vista
leerDouble()	double	public	String mensaje	Solicita y convierte un decimal; una entrada inválida	Vista

				puede producir NumberFormatException.	
leerTexto()	String	public	String mensaje	Solicita una línea de texto.	Vista
mostrarOrden()	void	public	OrdenServicio orden	Muestra todos los datos de una orden.	Vista
mostrarOrdenes()	void	public	List<OrdenServicio> ordenes	Muestra una colección de órdenes.	Vista
mostrarMensaje()	void	public	String mensaje	Muestra información o mensajes de error.	Vista
Controlador()	—	public	GestorOrdenes gestor, Vista vista	Inicializa el controlador con modelo y vista.	Controlador
iniciar()	void	public	Ninguno	Mantiene el menú activo hasta seleccionar Salir.	Controlador
procesarOpcion()	void	private	int opcion	Ejecuta la operación correspondiente a la opción elegida.	Controlador
registrarOrden()	void	private	Ninguno	Solicita datos, crea la orden y controla entradas/validaciones inválidas.	Controlador
buscarOrden()	void	private	Ninguno	Solicita número, busca y maneja una orden inexistente con try-catch y finally.	Controlador
modificarOrden()	void	private	Ninguno	Solicita nuevos datos y maneja errores de búsqueda o validación.	Controlador
cancelarOrden()	void	private	Ninguno	Solicita número y maneja el caso de orden inexistente.	Controlador
main()	void	public static	String[] args	Crea GestorOrdenes, Vista y Controlador; luego llama iniciar().	Main

Excepciones

Situación	Excepción	Lugar de manejo	Resultado
Entrada no numérica para número de orden, opción o costo	NumberFormatException	Controlador, alrededor de las lecturas numéricas	Mostrar error y volver al menú.
Costo <= 0 o texto obligatorio vacío	IllegalArgumentException	Constructor/setters o GestorOrdenes; capturada en Controlador	Informar dato inválido sin cerrar el programa.
Número de orden repetido	IllegalArgumentException	GestorOrdenes.agregarOrden(); capturada en Controlador	No agregar la orden y mostrar mensaje.
Buscar/modificar/cancelar una orden inexistente	NoSuchElementException	GestorOrdenes; capturada en Controlador	Informar que la orden no existe.
Solicitar mayor costo sin órdenes	NoSuchElementException	GestorOrdenes; capturada en Controlador	Informar que no hay órdenes registradas.
Finalización de una búsqueda	finally	Controlador.buscarOrden()	Mostrar siempre el cierre de la operación.

UML

